

0.6. La même ligne, avec une boucle while

La boucle for, décomposée en ses trois morceaux.

1. Ce qu'il faut comprendre

`while (i < 20)` répète **tant que** la condition est vraie. C'est la même boucle que `while (true)` du 0.0, mais celle-ci **s'arrête**.

Les trois morceaux que `for` réunit sur une ligne sont ici écrits **séparément** : le départ `uint8_t i = 0;` avant la boucle, la condition `i < 20` dans le `while`, et `i++;` à la fin de chaque tour.

Le piège : sans `i++`, `i` reste à 0, la condition reste vraie, et la boucle **ne s'arrête jamais**. Le programme reste bloqué dedans.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1333 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t i = 0;           // 1. on part de la colonne 0

    while (i < 20) {         // 2. tant qu'on est dans l'écran
        poser(i, 0, ALPHABET[i]); // la lettre n° i, dans la colonne i
        i++;                // 3. on passe à la suivante
    }

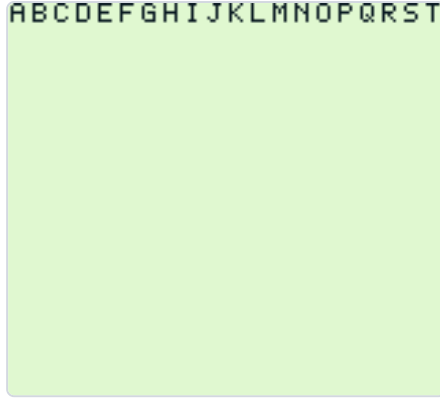
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur toute la première ligne, comme au 0.5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 est pleine : A à T
- ✓ rien ne déborde sur la ligne 1

Ensuite : **0.6.1. La même ligne, avec une boucle while — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++